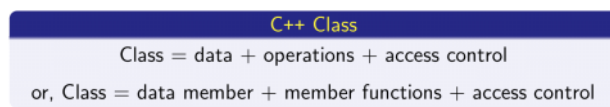
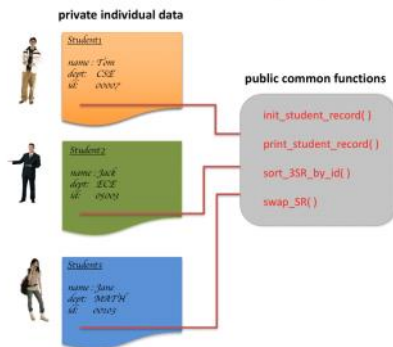


class

2024年11月20日 15:16

for struct->functions are global

- Factor out the **common** functions so that the compiler generates only **one copy** of machine codes for each function.
- But functions are “struct-specific” — they can only be called by objects of the intended struct. Now you get a **class**!



Class is also known as abstract data type (ADT).

In addition, C++ allows access control to each data member and member function:

public: accessible to any functions (both member functions of the class or other functions)

private: accessible only to member functions of the class enforce information hiding

protected

create a class for objects of the same type-> eg.cars

- An object is **constructed** when it is
 - **defined** in a scope (**static object** on stack).
 - **created** with the **new** operator (**dynamic object** on heap).
- An object is **deconstructed** when
 - it goes **out of a scope** (**static object**).
 - it is **deleted** with the **delete** operator (**dynamic object**).
- For “objects” of **basic data types** (actually they are **not** objects in C++), their construction and destruction are built into the C++ language.
- For (real) objects of **user-defined classes**, C++ allows the **class developers** to write their own construction and destruction functions: **constructors** and **destructor**.
- Besides creating an object, a constructor may also **initialize** its contents.
- A class may have **more than 1** constructor (function overloading), but it can only have **1 and only 1** destructor.

object is constructed when:

->defined in scope(stack栈)

->"new" operator(heap堆)

object is destructed when:

->out of scope
->delete

*基本数据类型如int, double是自动处理的

*自定义: 可以自己构造销毁

*一个class可以有多个构造函数, 只能有一个析构函数

如果没定义class的构造函数和析构函数, 会自动生成默认函数->不会为成员初始化, 只会为类的对象保存足够内存/释放对象占有内存 (不会删除new类内存)

```
temperature::temperature() { }  
temperature::~~temperature() { }
```

Destructor不是手动调用的! 是函数超作用域时自动调用

1. 如果对象是局部变量 (在栈上创建), 当对象离开其作用域时会自动调用

```
cpp  
  
void someFunction() {  
    Lamp myLamp; // 构造函数被调用  
    // ... 其他代码 ...  
} // 函数结束时, 析构函数~Lamp()自动被调用
```

2. 如果对象是用 new 创建的 (在堆上), 当使用 delete 删除对象时会自动调用

```
cpp  
  
Lamp* pLamp = new Lamp(); // 构造函数被调用  
delete pLamp; // 析构函数~Lamp()自动被调用
```

dynamic members are inadequate

data members are usually private->contents of the objects of the class

member functions are usually public->different objects of the class has the same interface

*data members and member functions are members of a class, accessed by "." operator

least 4 member functions

->constructor

->destructor

->accessor:inspect data member(not modifying it)

->mutator:will modify some data members

member functions can be defined:

->inside the class definition in a .h header file

->outside the class definition in a .cpp source file

-->the function name must be prepended with the class name and the

special class scope operator

```
// 在源文件中定义  
void MyClass::setValue(int newValue) {  
    value = newValue;  
}
```

```
int value = 10; // 这是全局变量
```

```
class MyClass {  
public:
```

```
    int value = 20; // 这是 MyClass 类的成员变量
```

```
    void printValue() {
```

```
        std::cout << "Local value: " << value << std::endl; // 访问类中的
```

成员(输出20)

```
        std::cout << "Global value: " << ::value << std::endl; // 访问全局
        变量(::没有前缀默认是访问全局变量, 输出10)
    }
};
```

```
int main() {
    MyClass obj;
    obj.printValue(); // 调用 printValue() 函数
    return 0;
}
```

!all data members should be private

!DON'T expose the difference between stored data and derived data

!DON'T expose a class' internal structure

!DON'T expose the implementation details of a class

- A **class** is a **user-defined** type representing a set of objects with the **same** structure and behavior.
- **Objects** are variables of a class type.
- **Instantiation**: The process of creating an object of a class is called **instantiating an object**.
- Each object of a class has its **own** copies and values of its **data members**.
- All objects of a class **share** a common set of **member functions**.
- To call a function, before we say
"call function X" or simply "call X".
- In OOP, we have to say
"invoke **method/operation/function** X on **object** Y of **class** Z".

Class Member Functions

- These are the functions **declared** inside the **body** of a class.
- They can be **defined** in two ways:

- ① **Within** the class body, then they are **inline functions**. The keyword **inline** is optional in this case.

```
class temperature
{
    ...
    double kelvin() const { return degree; }
    double celsius() const { return degree - 273.15; }
};
```

Or,

```
class temperature
{
    ...
    inline double kelvin() const { return degree; }
    inline double celsius() const { return degree - 273.15; }
};
```

COMP2011 (Fall 2024)

p.29

Class Member Functions ..

- ② **Outside** the class body, then add the prefix consisting of the class name and the **class scope operator ::**
(Any benefits of doing this?)

```
/* File: temperature.h */
class temperature
{
    ...
    double kelvin() const ;
    double celsius() const;
};
```

```
/* File: temperature.cpp */
#include "temperature.h"
...
double temperature::kelvin() const { return degree; }
double temperature::celsius() const { return degree - 273.15; }
```

COMP2011 (Fall 2024)

p.30

这两页PPT在讲解C++中inline(内联)函数的概念和特点。让我为你总结一下关键内容：

1. 函数调用的开销：

- 普通函数调用是"昂贵的"，因为操作系统需要在后台做很多工作来实现函数调用
- 包括保存现场、参数传递、跳转等操作

2. inline关键字的作用：

- 对于那些较小且经常被调用的函数，使用inline可能会更高效
- inline会将函数代码直接展开(unfold)到调用处，消除了函数调用的开销
- 比如原来是 `y = f(5)` 的函数调用会被直接展开成 `y = 4*5*5 + 9*5 + 1`

3. inline的优缺点： 优点：

- 避免了函数调用的开销，提高运行效率
- 保留了函数的良好特性（易读性、可维护性、类型检查）

缺点：

- 会增加程序的体积（源代码和可执行文件都会变大）
- 编译器可能不会遵守inline声明
- inline只是对编译器的一个"提示"(hint)，编译器有自由决定是否真的要内联这个函数
- 特别是当函数较大时，编译器可能会忽略inline声明

总的来说，这两页PPT在讲解如何在函数调用开销和代码体积之间做权衡，以及如何使用inline关键字来优化小型频繁调用的函数。

Inline Class Member Functions

- Class member functions defined **inside** the class definition body are automatically treated as **inline functions**.
- To enhance readability, one may also define them **outside** the class definition **but** in the **same** header file.

```
/* File: temperature1.h */
class temperature
{
    ...
    inline double
    kelvin() const
    {
        return degree;
    }
};
```

```
/* File: temperature2.h */
class temperature
{
    ...
    inline double kelvin() const;
};

inline double
temperature::kelvin() const
{
    return degree;
}
```

delete an array of user-defined objects using **delete[]**

delete [] will call the class destructor on each array element in reverse order: the last element **first** till the **first** one.